SMART INDIA HACKATHON 2026 | TEAM PRAXIS

ERSA — Sovereign On-Premise Agentic AI Workbench

Prototype Technical Brief & Presentation Defence Pack. Everything a judge is likely to probe — architecture, models and exact versions, the routing theory, the agent loop, every tool, the security/air-gap proof, and how each visible frontend feature is implemented.

---

1. One-line pitch & the problem

What it is: a single-node, fully on-premise, air-gapped *agentic* AI workbench for confidential industrial knowledge work — it plans and executes multi-step tasks (document search, verified computation, Word/Excel/PPT generation, vision, OCR, image generation, forecasting) using small open-weight models running locally, with **zero data ever leaving the organisation's network**.

Problem: industrial teams (refineries, plants) need AI for high-volume knowledge work over scanned inspection reports, P&IDs, SOPs and manuals, but that data is confidential. Public cloud AI violates data-confidentiality policy, creates an un-auditable compliance risk ("shadow AI"), and rising breach costs make it untenable. The need: a private, auditable, on-premise AI that runs on limited enterprise hardware.

> **Design thesis (the 4 differentiators in the deck):**
>
> 1. **Verified Generation** — any calculation is executed and verified in a sandbox before it is written into a Word/Excel report, so no hallucinated numbers.
> 2. **Explainable Routing** — a zero-GPU, weighted-keyword router picks the right model/tool and logs *why*; the routing decision is deterministic and reproducible.
> 3. **Domain Intelligence** — a LoRA-tuned small model specialised for a recurring document type (approval notes), with a pipeline to add more.
> 4. **Proven Security** — live network monitoring + an in-process egress firewall + a SHA-256 hash-chained audit log = demonstrable isolation and tamper-evidence for every task.

2. Architecture — single-node deployment

The system was deliberately collapsed from an earlier 3-service design (separate backend / agent / tools processes talking over localhost HTTP) into **one FastAPI process**. This is the core architectural decision and its rationale is the security story.

2.1 Processes on the box

Component	What it is	Network exposure
The app (`app/`, FastAPI + Uvicorn)	The ONE process: API routes, router, agent loop, all tools, inference client, audit log, security layer — all direct in-process Python calls.	`0.0.0.0:8000` — the **only** LAN-facing listener in the whole system.
Ollama	Third-party binary serving the local LLMs. Kept as a separate OS process because it is not our code.	Bound to `127.0.0.1:11434` only (loopback). Unreachable from the LAN.
Docker sandbox	Ephemeral, network-disabled container spawned per code-execution run. The container boundary IS the security isolation.	No network (`network_disabled=True`). One container per run, then destroyed.

Ports 8001 / 8002 from the old design no longer exist — a judge can verify with `curl 127.0.0.1:8001` → connection refused.

2.2 Request lifecycle (the "working")

1. **Submit** — browser POSTs `/api/submit-task` (or `/api/chat/{id}/message`). The API immediately returns `{task_id, status:"queued"}` and dispatches the work fire-and-forget (never blocks the HTTP response).
2. **Concurrency gate** — the task acquires a slot on one of two `asyncio.Semaphore` lanes: a general lane (`max_concurrent_tasks=2`) and a smaller **separate vision lane**, so a burst of heavy image inferences can't starve ordinary text tasks. Extra tasks genuinely queue.
3. **Route** — `router/` classifies the task type (no model call) and picks the entry model.
4. **Agent loop** — `agent/loop.py` runs *plan → act → observe → replan* until the planner says `finalize`, with a hard max-step cap so a planner bug can never hang a request.
5. **Act** — each step is either a model call (Ollama) or a tool call (in-process Python; code execution additionally shells to Docker).
6. **Finalize** — response is assembled to a fixed schema: `status, model_used, models_used[], task_type, result{type,text|file_url}, steps[], token_usage, error`.
7. **Audit** — one tamper-evident hash-chained row is written; the network monitor is sampled at task start and end and correlated by `task_id`.
8. **Poll** — browser polls `/api/task-status/{id}` until `completed` / `failed`.

3. Models used — exact identifiers & why

All model names are read from `config.json → models` (single source of truth; `router/model_registry.py` maps task types to these strings). The first three are served by Ollama; the last two are loaded directly from local weight directories.

Role	Model (exact)	Served by	Used for	Why this one
Text / reasoning / code-gen / content-prep	`qwen3:1.7b`	Ollama	text-generation, the reasoning step of vision tasks, code generation, doc-search answer synthesis, chat	Strong instruction-following at ~1.7B; fits alongside the vision model in a small VRAM budget.
Vision	`moondream`	Ollama	Any uploaded image — scene/description + "what is wrong in this photo" analysis	Tiny vision-language model (~1.8B); designed for describing images on modest hardware.
Domain adapter	`approval-note-lora` (*LoRA over* `qwen2.5:1.5b-instruct`)	Ollama (`Modelfile`: `FROM qwen2.5:1.5b-instruct` + `ADAPTER approval-note-lora.gguf`)	Every model call in the approval-note document/text flow	Fine-tuned on ~60 approval-note examples so output reliably follows the Subject / Findings / Recommendation / Approval-Status structure.
Text-to-image	`sd-turbo` (`stabilityai/sd-turbo`, fp16)	diffusers, local weights in `extra_models/sd-turbo`	"generate an image of…" requests	Distilled 1-step diffusion — one inference step produces a usable image, fast enough for a live demo on CPU or GPU.
Time-series forecasting	`moment-1-small` (`AutonLab/MOMENT-1-small`)	momentfm, local weights in `extra_models/moment-1-small`	"forecast the next N values: …" requests	A pretrained time-series foundation model; 512-step context, fixed forecast head; runs offline.

LoRA training details (defensible specifics): base `unsloth/Qwen2.5-1.5B-Instruct` loaded in 4-bit; PEFT LoRA with `r=16`, `lora_alpha=16`, `lora_dropout=0`, target modules `q_proj,k_proj,v_proj,o_proj`; 3 epochs, lr `2e-4`, batch 1 × grad-accum 8, `adamw_8bit`, bf16, `max_seq_length=1024`; exported to GGUF `q4_k_m` so Ollama loads it directly. Trained on Unsloth to fit a 4 GB-class GPU. At inference the adapter is always called at a **lower temperature (0.2)** for steadier, standardised output.

Note on versions: `inference/README.md` mentions `qwen2.5:1.5b-instruct` as the original text model; the current `config.json` ships `qwen3:1.7b` as the text model after an upgrade. The LoRA adapter's base remains `qwen2.5:1.5b-instruct` (what it was trained on).

4. The Router — explainable, zero-GPU classification

`app/router/classifier.py`. **No model call is ever used to classify** — it is deterministic weighted keyword/phrase scoring, which is what makes routing reproducible and auditable.

4.1 How scoring works

- Per task type there is a signal table of `(phrase, weight, group)`. Weight tiers: **5** = explicit unambiguous phrase ("execute this code", "generate a word document"); **3–4** = strong term; **1–2** = weak/ambiguous word ("calculate", "find", "run") — not enough alone.
- Matching uses `\b` word-boundary regex, not substring (fixes old bugs like "sop" matching inside "stopped", or "sum" inside "summer").
- Extra deterministic boosts: a real arithmetic expression (`+3` code-execution), a fenced code block (`+5`), a comma-separated numeric series (`+4` forecasting), a corpus record ID like `SOP-PTW-01` / `EMP-1042` (`+4` doc-search), an equipment tag *in a question frame* (`+3` doc-search).
- A task type must cross `MIN_CONFIDENT_SCORE = 3` to be accepted; otherwise it safely falls back to **text-generation** rather than guessing off one weak word.
- Vision is **not scored**: an uploaded image mime type deterministically forces `task_type = "vision"`. No prompt text can fake vision, and no text can override a real image.
- **Multi-step detection:** if ≥2 types qualify AND a connector phrase is present ("and then", "based on the findings"), it emits an ordered `workflow` list (e.g. `["doc-search","document-generation"]`); document-generation wins as the primary type because the deliverable is a file.
- Every routing decision logs the winning signals, all scores, confidence, and the multi-step flag (`[ROUTER] …`) — explainability without any chain-of-thought, because there is none at this stage.

4.2 Task types

`text-generation, vision, code-execution, document-generation, doc-search, image-generation, time-series-forecasting` — plus `chat`, which the router upgrades a plain question into *only* when it is inside a chat and has no actionable signal (so "generate a word doc…" inside a chat still generates the file instead of being swallowed into KB-search mode).

5. The Agent Loop — plan / act / observe / replan

`app/agent/loop.py` + `app/agent/planner.py`. `decide_next_step(state)` is called after **every** step, looking at the full observation history — this is what makes it genuinely agentic, not a fixed pipeline. Actions: `call_qwen`, `call_moondream`, `call_tool`, `finalize`.

Flow	Steps
text-generation	call_qwen → finalize
vision (plain)	call_moondream → finalize
vision (needs reasoning — "explain / what's wrong")	call_moondream (describe) → observe → call_qwen (reason over the description + original ask) → finalize
code-execution	call_qwen (generate a self-contained Python script — the raw user prompt is NEVER run as code) → validate it compiles → call_tool(execute_code) in Docker → observe verified stdout/exit_code → call_qwen (explain, grounded only in that output) → finalize. If no valid Python is produced, the task fails loudly (`CodeGenerationError`) — never falls back to running the prompt.
doc-search	call_tool(search_docs) → observe passages → call_qwen (answer strictly from passages, cite the source doc, or say "not in the knowledge base") → finalize
document-generation	[if computational] call_qwen (verification-code) → execute_code → build content directly from verified JSON; [else] call_qwen (content-prep → TITLE/## Heading plain text) → call_tool(generate_file) with the *prepared structured content*, never the raw prompt → finalize
image-generation / forecasting	call_tool(generate_image / forecast_timeseries) → (forecasting) call_qwen to present the already-computed numbers verbatim → finalize

Robustness details: tool calls are capped at `MAX_TOOL_ATTEMPTS = 2`; content-prep JSON/markdown re-samples once on failure; empty exception strings (e.g. httpx `ReadTimeout`) are converted to a real message so a task never silently "completes" blank; markdown emphasis (`**bold**`, `#`) is stripped in code after every model call because nothing downstream renders markdown; `repeat_penalty = 1.3` is set on every Ollama call to suppress the small model's verbatim-repetition loops.

6. Tools (6) — libraries, versions, isolation

Tool	Implementation	Key libraries (pinned)	Isolation / offline note
execute_code	Writes Qwen-generated Python into an ephemeral Docker container (`python:3.11-slim`) via a tar stream (no bind mount), runs it, returns stdout/stderr/exit_code.	`docker 7.1.0`	`network_disabled=True`, `mem_limit=256m`, `cpu_quota=0.5`, `pids_limit=64`, 15 s timeout, container killed+removed after. If Docker is down: graceful `exit_code 127` "sandbox unavailable", task still completes and the model is told NOT to guess the answer.
search_docs (RAG)	Local vector search over `tools/docs_corpus/` (sample SOPs, employee directory, asset register, policies) + anything dropped in `shared_files/`. Block-aware chunking keeps a whole record/clause in one chunk; over-fetch then local re-rank with an exact-identifier lexical bonus.	`chromadb 0.5.5` (default on-disk, no server), local ONNX `all-MiniLM-L6-v2` embeddings, `python-docx 1.1.2`	ChromaDB telemetry is **hard-disabled**: `posthog.capture` is monkey-patched to a no-op at import (the `anonymized_telemetry=False` setting alone is buggy in 0.5.5). Known one-time caveat: the ~79 MB embedding model must be pre-cached before going fully offline.
generate_file	Turns the prepared `{title, sections: [{heading,body}]}` shape into a real `.docx` / `.xlsx` / `.pptx` in `shared_files/`, served at `/files/<name>`.	`python-docx 1.1.2`, `openpyxl 3.1.5`, `python-pptx 0.6.23`	Fully offline, no headless-Office step. Multi-deliverable prompts ("word doc on X then excel of Y") are split so each content-prep call only sees its own fragment.
scan_document (OCR)	Tesseract OCR on an uploaded photo (handwritten notes, nameplates, inspection sheets). Raw OCR is passed to Qwen with an explicit "this is unverified OCR, flag garbled parts" instruction.	`pytesseract 0.3.13`, `Pillow 10.4.0` + system `tesseract-ocr`	CPU-only — zero GPU/VRAM cost (deliberate: the tight budget is VRAM). Separate from Moondream because a VLM paraphrases text; Tesseract reads pixels literally.
generate_image	SD-Turbo pipeline, 1 inference step, `guidance_scale=0.0`, fp16 weights, lazy-loaded on first use.	`diffusers 0.32.2`, `torch 2.14.0`, `transformers 4.46.3`	Weights local in `extra_models/sd-turbo`; no download at call time. Auto-selects CUDA if present else CPU.
forecast_timeseries	MOMENT-1-small forecasting head; short series is *tiled* (not zero-padded) up to the 512-step context so periodicity is preserved; horizon parsed from the prompt ("next 5 values").	`momentfm 0.1.4`, `torch 2.14.0`	`local_files_only=True`; explicit `.to(device)` so it actually uses the GPU when available. Model computes the numbers; Qwen only presents them.

PDF ingestion (`pdfplumber 0.11.4`, `pymupdf 1.24.11`) is a two-stage offline path: direct text extraction, then OCR fallback for scanned-only PDFs. Extracted PDF text is fed to the LoRA adapter *only* when the request is already an approval-note request.

7. Verified Generation — the anti-hallucination mechanism

The deck's headline claim. When a document-generation or code-execution request is *computational* (word-boundary match on `calculate, sum, average, fibonacci, prime, sequence, statistics…` or a regex for "first N numbers", "Nth prime", "sum of…"):

1. Qwen writes a Python script that computes the data and `print(json.dumps(result))`.
2. The script runs in the Docker sandbox — a **real Python interpreter**.
3. The file content is built **directly from that verified JSON in plain code** — Qwen is not asked to re-transcribe the numbers (that only adds a chance to hallucinate).
4. If the sandbox is unavailable or the script produced nothing, after one retry the task **fails loudly** (`VerifiedComputationError`) rather than let the model fill in the numbers.

Result: a number in a generated Excel/Word file was either computed by a real interpreter, or the task failed — it was never guessed by an LLM.

8. Security & Sovereignty — the three-layer proof

8.1 In-process egress firewall (enforcement)

`app/security/egress_firewall.py` — installed **before any router, DB or model client** can open a socket. It monkey-patches `socket.socket.connect / connect_ex` process-wide. Any outbound connection to a **publicly routable** address is refused at `connect()` time (raises `EgressBlocked`) and recorded in a ring buffer with the calling code frame. Allowed: loopback (incl. Ollama), RFC1918 / link-local / CGNAT, and operator-configured LAN peers. Endpoint `POST /api/egress-firewall/self-test` actively tries to reach 8.8.8.8:53, 1.1.1.1:443, `api.openai.com`, `huggingface.co` and returns `verdict: "ISOLATED"` when every one is blocked — this is the on-screen demo proof. Honest scope: this guards *this process*; `harden_network.sh` is the optional OS-level layer.

8.2 Network monitor (observation)

`app/monitor/network.py` — a `psutil` sweep of the machine's own ESTABLISHED TCP connections, classified LOCAL / LAN_CLIENT / EXTERNAL / VIOLATION. Sampled at each task's start and end and correlated by `task_id` (lock-guarded dict, no shared "current task" global, so concurrent tasks keep independent records). The UI phrases it honestly as "N external connections observed", never "zero calls ever".

8.3 Hash-chained audit log (tamper-evidence)

`app/audit/log.py` — SQLite. Every task writes one row: `task_id, user_id, task_type, model_used, timestamp, file_uploaded`, plus `prev_hash` and `entry_hash = SHA-256(payload | prev_hash)`. Altering any historical row breaks the chain from that point forward. `client_ip` and token counts are stored as *informational* columns, deliberately outside the hash payload so adding them didn't invalidate existing entries. Exposed at `GET /api/audit-log`.

8.4 Docker sandbox isolation

Covered in §6 — `network_disabled=True` is a sovereignty guarantee, not just a safety nicety; verifiable by asking the sandbox to open a URL and watching it fail.

9. RAG / Knowledge Base — three isolated collections

One ChromaDB (one client, one on-disk store, one embedding function) with **three collections**:

Collection	Scope key	Lifetime / reach
`mrpl_docs` (corpus)	filename	Auto-synced to `docs_corpus/` + `shared_files/` on every query — content-hash detects new/changed/deleted files, stale chunks removed. Reached via the explicit **doc-search** route.
`chat_kb`	`chat_id` (mandatory filter)	Per-chat uploads. A document uploaded in one chat can never be retrieved from another. Survives restart (chunks on disk) even though chat messages are in-memory only.
`global_kb`	`user_id` (mandatory filter)	Per-operator persistent KB — in retrieval scope for every chat that operator opens, until they explicitly delete it.

The chat flow uses `search_all()` — merges this chat's uploads + the operator's global KB, re-ranks by score, de-dupes, truncates to top-k. Citations (`{filename, page, score}`) are returned in the response `sources` field and shown in the UI. Answers are strictly grounded: "That information is not in the knowledge base." when passages don't contain it.

10. Frontend — features & how each is implemented

Vanilla HTML/CSS/JS, no build step. Only external loads are Google Fonts + the Iconify web component. Served from the same `:8000` origin, so "one LAN URL" is literally true. Files: `index.html` / `user.js` (workbench, 2.4k LOC), `admin.html` / `admin.js` (ops console), `login/signup/admin-login`, `app.js` (shared), `styles.css`.

Visible feature	How it works
Live execution graph (Classifying → Routing → Processing → Validation → Result, with a Document-Search branch)	Driven by the same state machine the backend uses. While `processing` the graph shows an *honest generic* progression because `/api/task-status` doesn't reveal `task_type` mid-flight; on `completed` it reconciles with the real `model_used`, `result.type` and the `steps[]` trace — branches not taken stay visible but dimmed, and a text task's tool node is marked "not confirmed" rather than guessed.
Node inspector popover	Hover to peek, click to pin; anchored to the graph node. Reads the per-step trace (`action, model_used, tool_name, prompt/completion tokens`) from the task-status response.
Document preview panel (slides in from the right)	`app/tools/doc_preview.py` renders server-side, fully offline: PDF embeds raw; `.docx/.xlsx/.pptx` parsed with the same python-docx/openpyxl/python-pptx libs into escaped HTML (headings, tables, one block per slide); `.txt/.md` as text. No headless Office. Endpoints `/api/preview/generated/{file}`, `/api/kb/{id}/preview`.
Persistent Knowledge Base manager	Add / remove / search files that stay in retrieval scope for every chat — backed by the `global_kb` collection; endpoints `/api/kb/upload`, `/api/kb/list`, `/api/kb/{id}` (DELETE), `/api/kb/{id}/raw`.
Multi-file attach + chat upload	`/api/chat/{id}/upload` → PDF pages extracted (`pdf_ingest`) → chunked per page → embedded into `chat_kb` with page numbers for citations.
Voice input	Browser-native Web Speech API (`SpeechRecognition`) — no server component, no external service.
Task-template bar + template library	Client-side prompt presets that populate the composer.
Notifications (bell)	Browser `Notification` API only, gated on the existing status polling — fires once on the first transition to `completed` / `failed`; degrades gracefully if permission denied.
Theme toggle + disco mode + gradient colour picker	CSS custom-property design tokens (dark default + light theme) swapped on `:root`; preferences in `localStorage`.
Login / signup / dynamic nickname greeting	Client-side account store in `localStorage`; greeting interpolates the saved nickname. (Not server-side auth — see limitations.)
Admin "Sovereign AI Operations" console	Overview, Users, Task Activity, Model & Routing, Knowledge Base (all uploads across chats with titles), Audit Log (`/api/audit-log`), Security & Sovereignty (egress-firewall status + self-test), System Resources. Admin gate is a client-side passcode (`AdminAuth` in `app.js`, `localStorage` / `sessionStorage`) — intentionally not claimed as real auth.

11. Full technology stack (pinned)

Backend / runtime

- Python 3.11, `fastapi 0.115.0`, `uvicorn[standard] 0.30.6`, `httpx 0.27.2`, `pydantic 2.9.2`
- `python-dotenv 1.0.1`, `psutil 6.0.0`
- Ollama (LLM server, localhost only)
- Docker + `docker 7.1.0` SDK, image `python:3.11-slim`

ML / inference

- `torch 2.14.0`, `transformers 4.46.3`, `accelerate 1.14.0`, `safetensors 0.8.0`
- `diffusers 0.32.2` (SD-Turbo), `momentfm 0.1.4` (MOMENT-1-small)
- Unsloth + TRL + PEFT (offline, LoRA training only)

Retrieval / documents

- `chromadb 0.5.5` + ONNX `all-MiniLM-L6-v2` embeddings
- `python-docx 1.1.2`, `openpyxl 3.1.5`, `python-pptx 0.6.23`
- `pdfplumber 0.11.4`, `pymupdf 1.24.11`
- `pytesseract 0.3.13` + system `tesseract-ocr`, `Pillow 10.4.0`

Frontend

- Vanilla HTML5 / CSS3 / ES-modules JavaScript, no build step
- Google Fonts + Iconify web component (only CDN loads)
- Web Speech API, Notification API, localStorage/sessionStorage

Transparency / DB

- SQLite hash-chained audit log (`hashlib` SHA-256)
- `pytest 8.3.3` + `pytest-asyncio 0.24.0` — ~53 tests

12. Known limitations (state these honestly if asked)

- **Small-model reliability:** Qwen / the LoRA adapter occasionally answer from general knowledge instead of retrieved passages, or vary specifics in generated docs. Retrieval and verified-computation are correct; the variance is model behaviour. Re-running usually cleans it up.
- **Multi-step chaining:** the classifier exposes an ordered `workflow` for "search X then generate a doc", but the planner does not yet fully feed doc-search results into the document content — the request classifies correctly and produces a coherent doc, but grounding across the two steps is a flagged gap.
- **OCR:** printed/typed text only (Tesseract). Cursive handwriting is weak and labelled as such — don't demo a handwriting scan.
- **Auth:** login/signup/admin-gate are client-side only (`localStorage`). Real server-side auth is a single well-isolated module (`AdminAuth`) away; the locked API contract had no session endpoint.
- **Chat history** is in-memory (conversational continuity, not permanent storage); uploaded-document chunks persist on disk.
- **One-time online step:** the ~79 MB Chroma embedding model must be cached once before the machine goes fully air-gapped.

13. Anticipated judge questions — crisp answers

Q. Is this actually agentic or just a pipeline?

Agentic. `decide_next_step(state)` is re-invoked after every step against the full observation history and can branch, retry a tool (cap 2), or add a reasoning step — e.g. a vision "what's wrong here" request dynamically chains Moondream → Qwen. A hard max-step cap bounds it.

Q. How do you guarantee no data leaves the network?

Three layers: (1) an in-process egress firewall that refuses every off-LAN `connect()` and has a live self-test that tries OpenAI/HuggingFace/DNS and shows them all BLOCKED; (2) a psutil network monitor correlated per task; (3) the code sandbox runs `network_disabled=True` . Ollama is loopback-only. The only non-loopback listener is `:8000` .

Q. How do you prevent hallucinated numbers in reports?

Verified Generation: for any computational request, Qwen writes a Python script, it runs in a real interpreter in the sandbox, and the file is built directly from that verified JSON in plain code — the LLM never re-types the numbers. If verification fails, the task fails rather than guess.

Q. Why keyword routing instead of an LLM router?

Determinism, zero GPU cost, and explainability — the router logs exactly which weighted signals fired and all scores. An LLM router would be slower, non-reproducible, and consume the VRAM budget we need for the actual task. Weak words can't trigger a route alone (threshold = 3); ambiguity falls back to plain text.

Q. What is the LoRA adapter and is it real?

Yes. LoRA (`r=16`) over Qwen2.5-1.5B-Instruct, trained with Unsloth on ~60 approval-note examples, 3 epochs, exported to GGUF q4_k_m, loaded by Ollama via a `Modelfile` with `ADAPTER` . It reliably produces the Subject/Findings/Recommendation/Approval-Status structure; called at temp 0.2. A pipeline exists to add adapters for other document types.

Q. What hardware does this need?

A single machine with one mid-range GPU (design target ~4–8 GB VRAM; the current admin spec lists an RTX 4060). Text + vision models co-reside in VRAM; OCR and audit run on CPU on purpose to protect the VRAM budget. Concurrency is capped at 2 general + a small vision lane.

Q. What if Docker / Ollama / OCR is not available?

Each degrades gracefully with an honest error and the model is instructed not to fabricate a result: no Docker → `exit_code 127` "sandbox unavailable"; no tesseract → "OCR not available on this machine"; a model timeout → a real error message, never a blank "completed".

Q. How is the audit log tamper-evident?

Each row's `entry_hash = SHA-256(task_id|user_id|task_type|model_used|timestamp|file_uploaded|prev_hash)` . Because `prev_hash` is the previous row's hash, editing any historical row breaks every subsequent hash — detectable by re-walking the chain.

Q. Is the "SECURE / zero external calls" claim overstated?

We phrase it as evidence, not a permanent guarantee: the UI says "N external connections observed in this sweep". The enforcement (egress firewall) is the hard guarantee for this process; Wireshark / Resource Monitor remain the independent packet-level check.

Q. How do multiple users on the LAN not see each other's data?

Chat KB is filtered by a mandatory `chat_id` ; the global KB by a mandatory `user_id` . Every query carries the filter — a cross-chat or cross-user retrieval is structurally impossible, not just discouraged. The audit log records the real TCP `client_ip` Starlette saw (not a client-claimed value).

Q. Why single-node instead of microservices?

Attack surface and provability. One process = one LAN port to reason about, one thing to firewall, direct function calls instead of inter-service HTTP hops. The only intentional external boundaries (Ollama binary, the per-run sandbox container) are exactly the two that must be separate.

Q. Can it add new document types or models later?

Models: one line in `config.json` (single source of truth). New tool: one module + one facade wrapper + one `tool_dispatch` entry + one planner branch (that's exactly how image-gen and forecasting were added on top of the original 3-tool contract). New LoRA: reuse the Unsloth → GGUF → Modelfile pipeline.

Team PRAXIS · ERSA — Sovereign On-Premise Agentic AI Workbench · Smart India Hackathon 2026. Prepared from the prototype source (single-node `app/` build, branch `integration-test`). Model identifiers, library versions and mechanisms in this brief are taken directly from `config.json`, `requirements.txt` and the implementation.